GEO5017 A2: Object Classification from Airborne LiDAR Data

Henryk Gujda Adriano Landman
Mathijs vander Stichele

Delft University of Technology, Faculty of Architecture and the Built Environment

March 2026

1 Introduction

The goal of this assignment is the classification of 500 point clouds. In other words, understanding and assigning each point cloud a meaningful semantic label within five categories: *building*, *car*, *fence*, *pole*, and *tree*. We design and evaluate discriminative geometric features, perform feature selection, and train and compare Support Vector Machine (SVM) and Random Forest (RF) classifiers.

2 Methodology

The first step is engineering a set of geometric features from the raw point cloud coordinates, then applying a forward search to identify the best subset of features. Then, training and evaluation of two classifiers namely, Support Vector Machine (SVM) and Random Forest (RF) comparing their performance with different hyperparameter settings.

2.1 Feature Design

About 26 geometric features were researched, then 10 implemented and evaluated from the designed feature set: ['height', 'root density', 'area', 'shape index', 'linearity', 'sphericity', 'slenderness', 'length height ratio', 'circularity', 'footprint density']

1. **Height** x_1: maximum z-coordinate of all points, $x_1 = \max_i z_i$. Tall structures (buildings, poles) are separated from low ones (cars).
2. **Root density** x_2: fraction of points within radius r=0.2 m of the lowest point in 2D, $x_2 = |\{p : \|p_{xy} - p_{xy}^{\text{root}}\| \leq r\}|/N$. Dense roots indicate poles or tree trunks.
3. **Area** x_3: area of the 2D convex hull, $x_3 = \text{Vol}(\mathcal{H}_{2D})$. Buildings have large footprints; poles are small.
4. **Shape index** x_4: compactness ratio $x_4 = x_3/\text{Perim}(\mathcal{H}_{2D})$. Circular objects (trees, poles) score high; elongated fences score low.
5. **Linearity** x_4: compactness ratio $x_4 = (\lambda_3 - \lambda_2)/\lambda_3$ where λ_3 is the largest and λ_2 second largest eigenvector. difference between the two largest eigenvalues divided by the largest. A large value of linearity indicates an object is long and slender.
6. **Sphericity** $x_6 = \lambda_3/\lambda_1$. Is the ratio of the smallest eigenvector to the largest. Circular objects (trees, poles) score high and are; elongated fences, roofs and poles score low.
7. **Slenderness** x_7: ratio of height to the square root of hull area, $x_7 = x_1/\sqrt{x_3}$. Poles score very high; buildings and cars score low.
8. **Length–height ratio** x_8: ratio of the longest 2D convex-hull edge ℓ to height, $x_8 = \ell/x_1$. Fences are long and flat (high ratio); poles are tall and narrow (low ratio).
9. **Circularity** x_9: ratio of hull area to the area of the minimum bounding circle of radius r_c, $x_9 = x_3/(\pi r_c^2)$. Trees and poles have near-circular footprints; fences and buildings do not.
10. **Footprint density** x_{10}: number of points per unit hull area, $x_{10} = N/x_3$. Dense, compact objects (cars, poles) score high relative to sparse ones (fences, large building roofs).ed fences score low.

2.2 Feature Selection

We evaluate feature discriminability using forward search. The within-class and between-class scatter matrices are defined as

$$S_W = \sum_{k=1}^{K} \frac{N_k}{N} \Sigma_k, \qquad S_B = \sum_{k=1}^{K} \frac{N_k}{N} (\mu_k - \mu)(\mu_k - \mu)^T,$$

and feature quality is measured by

$$J = \frac{\mathrm{tr}(S_B)}{\mathrm{tr}(S_W)}.$$

Here, K denotes the number of classes, N_k the number of samples in class k, N the total number of samples, μ_k the class mean, μ the overall mean, and Σ_k the covariance matrix of class k.

Starting from the empty feature set, one feature is added at each step such that the resulting subset achieves the highest value of J, until the desired number of features is selected . In this way, the selected subset maximizes class separability at each step while avoiding an exhaustive search over all feature combinations. This procedure selected the four features **height**, **circularity**, **length_height ratio**, and **root density**. The result indicates that the most discriminative information in the initial feature set is mainly related to object height, footprint shape, horizontal-to-vertical proportion, and the local point concentration near the base. Table 1 lists the scores for the 4 most effective features.

Table 1: Feature selection scores.

ID	Feature	Score
1	height	J = 2.08
2	circularity	J = 1.95
3	length height ratio	J = 1.79
4	root density	J = 1.66

3 Experiments and Evaluation

3.1 Hyperparameter Analysis

SVM. We tested three kernel functions (*linear*, *RBF*, *polynomial*) across regularization values $C \in \{0.1, 1, 10, 100\}$, kernel coefficient values $\gamma \in \{\texttt{'scale'}, 0.01, 0.03, 0.05\}$ for the *RBF* and *polynomial* kernels and degree values $d_{\text{poly}} \in \{1, 2, 3\}$ for the *polynomial* kernel. Table 2 shows the overall accuracy (OA) of the iteration process for only the different kernel functions and regularization values. The values for $\gamma = \texttt{'scale'}$ and $\text{degree} = 3$ are kept at default value throughout this iteration process. Therefore, Table 2 serves only to illustrate the iteration process and does not provide the complete results of the hyperparameter experimentation. The RBF kernel with C=**50** and γ=**0.03** is one of the models that achieves the best trade-off between test accuracy OA=**0.965** and the hyperparameter values and is adopted as the final SVM model.

Table 2: SVM OA (%) for different kernels and C values.

Kernel	C=1	C=10	C=50	C=100
Linear	0.965	0.965	0.965	0.965
RBF	0.940	0.955	0.960	0.960
Polynomial	0.920	0.960	0.960	0.960

Random Forest. We varied the number of trees $T \in \{10, 50, 100, 200\}$, maximum depth $d_{\text{tree}} \in \{5, 10, 20, \text{None}\}$, minimum samples required to split an internal node $n_{\text{split}} \in \{2, 3, 5\}$, minimum samples required at a leaf node $n_{\text{leaf}} \in \{1, 2\}$, and the number of features considered at each split $m_{\text{try}} \in \{\texttt{sqrt}, \texttt{log2}\}$. table 3 shows the performance of the classifier for the different values of T and d_{tree}, with the other hyperparamaters set to their default value. Various paramater combinations reach optimal performance based on OA. The final RF model uses T=**50**, d=**5**, n_{split}=**3**, n_{leaf}=**1** and m_{try}=**sqrt**.

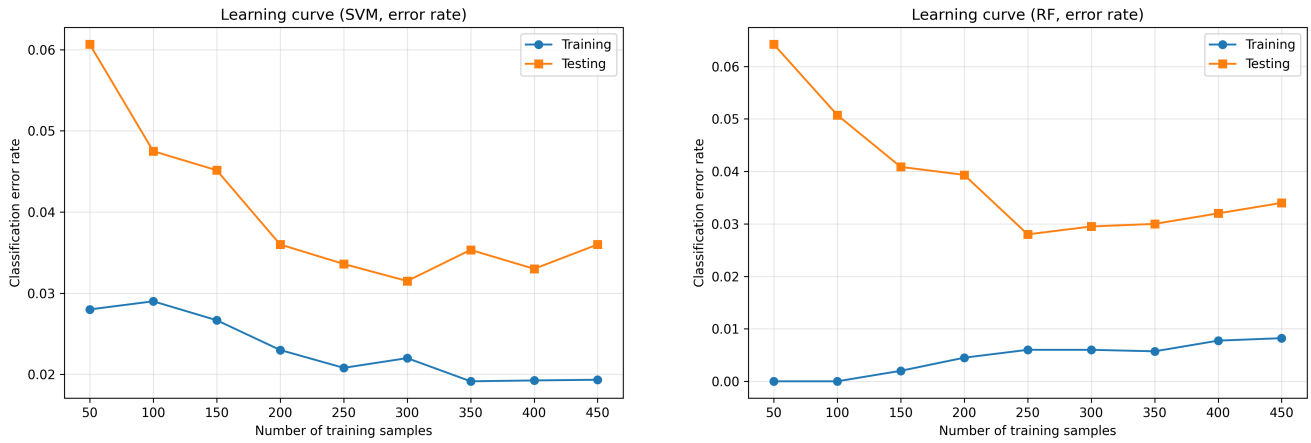
Table 3: RF OA (%) for different T and d_{tree} values.

d_{tree}	T=10	T=50	T=100	T=200
5	0.970	0.970	0.970	0.970
10	0.950	0.970	0.970	0.970
20	0.950	0.970	0.970	0.970
None	0.950	0.970	0.970	0.970

3.2 Learning Curves

Learning curves were generated for the final SVM and RF models by gradually increasing the number of training samples and evaluating the classification error on both the training and testing sets, shown in Figure 1. Both classifiers overall benefit

from additional training data, as reflected by the decreasing test error with increasing training-set size. The SVM shows a narrowing gap between training and testing error, indicating improving generalization and limited overfitting at larger sample sizes. On the other hand, the Random Forest classifier maintains an almost zero training error across all training sizes, while the testing error remains noticeably higher. This indicates a stronger tendency to overfit. Although RF performs slightly better with very small training sets, the SVM achieves lower test error once a moderate number of training samples is available and therefore exhibits better generalization behaviour overall. We can see, for example, in the SVM learning curve that a training size of 300 results in a relatively low test error and a relatively small gap between test and train error. For RF, this is the case for a training sample of 300. Essentially, this is what is desired.



(a) SVM

(b) RF

Figure 1: Learning curves of the final SVM and RF models. The x-axis shows the number of training samples and the y-axis shows the classification error rate for both the training and testing sets.

3.3 Error Analysis and Comparison

Since the dataset is balanced across the five classes, overall accuracy is an appropriate primary metric for comparing the classifiers. To complement OA, macro-F1 is reported too, because it gives equal importance to each class and better reflects class-specific weaknesses. The confusion matrix is used for detailed error analysis.

The confusion matrices of both classifiers are shown by table 4. Confusion matrices are used to analyse where the classifiers perform well and where the remaining errors occur. Both classifiers perform very well on the classes **car** and **pole**, for which a recall of 1.00 is achieved in both models. The classes **building** and **tree** are also classified accurately, with only one building misclassified as fence and two trees misclassified as pole. The most difficult category is clearly **fence**. For SVM, fence has the lowest recall of 0.900, with three fence objects misclassified as car and one as pole. RF improves this slightly to a recall of 0.925 by correctly classifying one additional fence sample. The dominant confusion pattern in both models is therefore **fence** $\rightarrow$ **car**, followed by **tree** $\rightarrow$ **pole**. These errors are plausible given the selected feature set (*height*, *circularity*, *length–height ratio*, and *root density*). Fences and cars can overlap in low-height and footprint-based descriptors, while trees and poles both exhibit strong vertical structure and a concentrated base region. As a result, these category pairs are more difficult to separate than buildings or poles.

(a) SVM

	B	C	F	P	T
B	39	0	1	0	0
C	0	40	0	0	0
F	0	3	36	1	0
P	0	0	0	40	0
T	0	0	0	2	38

(b) RF

	B	C	F	P	T
B	39	0	1	0	0
C	0	40	0	0	0
F	0	3	37	0	0
P	0	0	0	40	0
T	0	0	0	2	38

Table 4: Confusion matrices for the final SVM and RF models. Rows denote ground truth and columns denote predictions. Class abbreviations: B = building, C = car, F = fence, P = pole, T = tree.

Using OA as the primary metric, the **RF classifier performs slightly better**, achieving an OA of 97.0% compared with 96.5% for SVM, i.e. an improvement of 0.5 percentage points. The same conclusion is reached when considering macro-F1, where RF scores 0.971 and SVM scores 0.967. Therefore, although both classifiers are highly effective, RF provides the best overall performance on the selected feature subset. Table 5 summarises the quantitative comparison.

Table 5: Classification results on the test set.

Classifier	OA (%)	Macro-F1
SVM	96.5	0.967
RF	97.0	0.971

4 Conclusion

This report investigated the classification of 500 airborne LiDAR objects into the five classes *building*, *car*, *fence*, *pole*, and *tree*. From an initial set of ten engineered geometric features, forward search based on the scatter criterion selected the four most discriminative descriptors: *height*, *circularity*, *length–height ratio*, and *root density*. Both classifiers achieved strong results on the selected feature subset. The final SVM model, using an RBF kernel with $C = 50$ and $\gamma = 0.03$, reached an overall accuracy of 96.5% and a macro-F1 score of 0.967. The final Random Forest model, using $T = 50$, $d_{\text{tree}} = 5$, $n_{\text{split}} = 3$, $n_{\text{leaf}} = 1$, and $m_{\text{try}} = \texttt{sqrt}$, performed slightly better with an overall accuracy of 97.0% and a macro-F1 score of 0.971. Error analysis showed that both models classified *car* and *pole* particularly well, while *fence* was the most challenging category. Overall, the Random Forest provided the best performance, although the SVM showed smoother learning behaviour and stronger generalization as the number of training samples increased.

Suggestions for improvement. Several directions could further improve the results. First, additional discriminative features could be introduced, such as 3D convex-hull volume or eigenvalue-based features at multiple neighborhood scales. But, it is important to be cautious, as a too complex classifier or a too enriched feature combination will result in severe overfitting (curse of dimensionality). Second, PCA could be performed to reduce redundancy by transforming correlated features into a smaller set of components. Finally, more advanced classifiers, such as gradient-boosted trees or deep-learning approaches for point clouds, could be explored to capture more complex geometric patterns directly from the data.

Contributions. **Henryk**: Introduction, Method, Feature engineering, Report Structure.
Mathijs: Feature Engineering, Classifiers, Learning curves, Classifier Evaluation
Adriano: Forward search, Hyperparameter Analysis, Learning curves, Conclusion

GitHub Repository Repository Link: `https://github.com/henq98/A2-Classification/tree/main`

References

[1] C.-H. Lin et al., "Eigen-feature analysis of weighted covariance matrices for LiDAR point cloud classification," *ISPRS J. Photogramm. Remote Sens.*, vol. 94, pp. 70–79, 2014.

[2] H. Thomas et al., "Semantic classification of 3D point clouds with multiscale spherical neighbourhoods," in *Proc. 3DV*, pp. 390–398, 2018.

[3] W. Gao et al., "SUM: A benchmark dataset of semantic urban meshes," *ISPRS J. Photogramm. Remote Sens.*, vol. 179, pp. 108–120, 2021.